

PostgreSQL Architecture & Schema Design Interview Guide

Table of Contents

1. [How to Use This Guide](#)
 2. [Interview Framework](#)
 3. [Schema Design Questions](#)
 4. [Indexing and Performance Questions](#)
 5. [Scaling and Architecture Questions](#)
 6. [Transactions and Consistency Questions](#)
 7. [Advanced PostgreSQL Features Questions](#)
 8. [System Design with PostgreSQL Questions](#)
 9. [Behavioral/Situational Questions](#)
 10. [Quick Reference: Key Numbers to Know](#)
 11. [Common Pitfalls to Avoid](#)
 12. [Cross-References](#)
-

How to Use This Guide

This guide contains 35+ database architecture and schema design interview questions with detailed answers. Each answer follows a structure:

- **Core answer** (what to say in the first 30 seconds)
- **Depth** (technical details to add if the interviewer wants more)
- **Example** (a concrete scenario or SQL snippet)
- **Follow-up** (what the interviewer likely asks next)

Questions are grouped by topic and difficulty. Aim to answer each question in 2–4 minutes unless asked to elaborate.

Interview Framework

Before answering any system design question, follow this framework:

1. CLARIFY requirements (2 min)
 - Scale: how many users, QPS, data volume?
 - Features: which are MVP, which are nice-to-have?
 - Constraints: latency requirements? consistency requirements?
2. ESTIMATE capacity (2 min)
 - Compute QPS, storage, bandwidth
 - State assumptions out loud
3. DESIGN schema (10 min)
 - Core tables, relationships, constraints
 - Draw ER diagram or describe entities
 - Call out tradeoffs (normalization vs. denormalization)

4. INDEX strategy (3 min)
 - Which columns? Which index type?
 - Partial indexes for common queries
5. SCALING discussion (5 min)
 - Start with what fits in one PostgreSQL instance
 - Identify bottlenecks and when they occur
 - Propose solutions: partitioning, read replicas, sharding
6. TRADEOFFS (2 min)
 - Every decision has a cost
 - Show you understand both sides

Schema Design Questions

Q1: Design a schema for a Twitter-like system.

Core answer:

```
CREATE TABLE users (  
    user_id    BIGSERIAL PRIMARY KEY,  
    username   VARCHAR(50) NOT NULL UNIQUE,  
    display_name VARCHAR(200),  
    bio        TEXT,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE tweets (  
    tweet_id    BIGSERIAL PRIMARY KEY,  
    user_id     BIGINT NOT NULL REFERENCES users,  
    body        VARCHAR(280) NOT NULL,  
    reply_to_id BIGINT REFERENCES tweets,  
    retweet_of_id BIGINT REFERENCES tweets,  
    like_count  INT NOT NULL DEFAULT 0,  
    retweet_count INT NOT NULL DEFAULT 0,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE follows (  
    follower_id BIGINT NOT NULL REFERENCES users,  
    followee_id BIGINT NOT NULL REFERENCES users,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
    PRIMARY KEY (follower_id, followee_id),  
    CHECK (follower_id != followee_id)  
);  
  
CREATE TABLE likes (  
    user_id     BIGINT NOT NULL REFERENCES users,  
    tweet_id    BIGINT NOT NULL REFERENCES tweets,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
PRIMARY KEY (user_id, tweet_id)
);
```

Depth: The hardest part is the **timeline (home feed)** — showing tweets from all accounts a user follows, sorted by recency. Two approaches:

- *Pull:* `SELECT t.* FROM tweets t JOIN follows f ON f.followee_id = t.user_id WHERE f.follower_id = $1 ORDER BY t.created_at DESC LIMIT 50` — works up to ~5K follows, then too slow.
- *Push (fanout-on-write):* Maintain a `timelines` table; when a tweet is posted, insert into every follower's timeline. Twitter uses a hybrid (push for normal users, pull for celebrities with 10M+ followers).

Follow-up: "What index do you put on tweets?" `CREATE INDEX ON tweets (user_id, created_at DESC)` — supports timeline pull queries. `CREATE INDEX ON tweets (created_at DESC)` — for global trending queries.

Q2: Design a schema for a hotel booking system.

Core answer:

```
CREATE TABLE hotels (hotel_id UUID PK, name, address, star_rating, ...);
CREATE TABLE room_types (type_id UUID PK, hotel_id FK, name, capacity, base_price, ...);
CREATE TABLE rooms (room_id UUID PK, hotel_id FK, type_id FK, floor, number, ...);
CREATE TABLE rates (rate_id UUID PK, type_id FK, date, price, min_stay, ...);
CREATE TABLE bookings (
    booking_id UUID PK,
    room_id UUID FK,
    guest_id UUID FK,
    check_in DATE,
    check_out DATE,
    status ENUM('confirmed', 'checked_in', 'checked_out', 'cancelled'),
    total_price NUMERIC,
    ...
);
```

Depth: The critical constraint is **no double-booking**. Use a PostgreSQL exclusion constraint with the `btree_gist` extension:

```
CREATE EXTENSION btree_gist;
ALTER TABLE bookings ADD CONSTRAINT no_double_booking
EXCLUDE USING GIST (
    room_id WITH =,
    daterange(check_in, check_out, '[') WITH &&
)
WHERE (status NOT IN ('cancelled'));
```

This creates a GiST index that prevents overlapping date ranges for the same room. This is more correct than application-level checks, which have race conditions.

Follow-up: "How do you handle pricing for different seasons / promotions?" Answer: A `rates` table with `(type_id, date, price)` where a billing job computes the total as `SUM(price) FOR each date in (check_in, check_out)` .

Q3: How would you model a permission system (RBAC)?

Core answer:

```
-- Resources and actions
CREATE TABLE permissions (
  perm_id SERIAL PK,
  resource VARCHAR(100), -- 'invoice', 'user', 'report'
  action VARCHAR(50), -- 'read', 'write', 'delete', 'admin'
  UNIQUE (resource, action)
);

-- Roles aggregate permissions
CREATE TABLE roles (role_id SERIAL PK, name VARCHAR(100) UNIQUE, ...);
CREATE TABLE role_permissions (
  role_id INT REFERENCES roles,
  perm_id INT REFERENCES permissions,
  PRIMARY KEY (role_id, perm_id)
);

-- Users are assigned roles (optionally scoped to an org/resource)
CREATE TABLE user_roles (
  user_id UUID REFERENCES users,
  role_id INT REFERENCES roles,
  scope_id UUID, -- NULL = global; set = scoped to that org/resource
  PRIMARY KEY (user_id, role_id, scope_id)
);
```

Depth: For dynamic ABAC (Attribute-Based Access Control), add a `conditions JSONB` column to `role_permissions` storing rule expressions (e.g., `{"field": "owner_id", "operator": "eq", "value": "$user_id"}`). The application evaluates these conditions at runtime.

Follow-up: "How do you check if a user has a permission efficiently?"

```
SELECT EXISTS (
  SELECT 1 FROM user_roles ur
  JOIN role_permissions rp ON rp.role_id = ur.role_id
  JOIN permissions p ON p.perm_id = rp.perm_id
  WHERE ur.user_id = $user_id
  AND p.resource = $resource
  AND p.action = $action
  AND (ur.scope_id IS NULL OR ur.scope_id = $scope_id)
);
```

Cache this result in Redis per `(user_id, resource, action, scope_id)` with a 5-minute TTL.

Q4: Design a schema for a hospital appointment system.

Core answer: Core entities: `patients`, `doctors`, `departments`, `availability_slots`, `appointments`.

The key insight is that availability is **time-slotted** and a doctor can only be in one appointment at a time:

```
CREATE TABLE availability_slots (  
    slot_id      UUID PK,  
    doctor_id    UUID FK,  
    slot_start   TIMESTAMPTZ NOT NULL,  
    slot_end     TIMESTAMPTZ NOT NULL,  
    is_booked    BOOLEAN DEFAULT FALSE,  
    EXCLUDE USING GIST (  
        doctor_id WITH =,  
        tstzrange(slot_start, slot_end, '[]') WITH &&  
    )  
);  
  
CREATE TABLE appointments (  
    appt_id      UUID PK,  
    slot_id      UUID UNIQUE FK, -- one appointment per slot  
    patient_id   UUID FK,  
    status       ENUM('scheduled', 'confirmed', 'cancelled', 'completed', 'no_show'),  
    notes        TEXT,  
    created_at   TIMESTAMPTZ DEFAULT now()  
);
```

Follow-up: "How do you handle cancellations and rebooking?" Cancelling sets `appointments.status = 'cancelled'` and `availability_slots.is_booked = FALSE` in a transaction. A background job can then notify waitlisted patients.

Q5: What is the difference between 3NF and BCNF? When do you denormalize?

Core answer:

- **3NF** (Third Normal Form): No transitive dependencies. Every non-key column depends only on the primary key. Example: storing `city_name` and `zip_code` in the same table violates 3NF if `city_name → zip_code`.
- **BCNF** (Boyce-Codd Normal Form): Stricter — every determinant must be a superkey. A table can be in 3NF but not BCNF when there are multiple overlapping candidate keys.

When to denormalize:

1. When a query that runs millions of times per day requires 3+ table JOINS to produce commonly needed data.
2. When aggregate values (counts, sums) are computed repeatedly from large tables (store the aggregate).
3. When sorting/filtering on a computed value (store the computed value, keep it in sync via trigger).
4. When serving pre-computed data from a cache or materialized view isn't fast enough.

Rule of thumb: Normalize for writes; denormalize for reads. Start normalized and only denormalize when you have measured evidence of a query performance problem.

Q6: How would you design a tagging system that scales?

Core answer: Three common approaches:

1. **Normalized M:N table** (most common):

```
CREATE TABLE tags (tag_id SERIAL PK, name VARCHAR(100) UNIQUE);
CREATE TABLE post_tags (post_id UUID, tag_id INT, PRIMARY KEY (post_id, tag_id));
CREATE INDEX ON post_tags (tag_id); -- for "all posts with tag X"
```

2. **PostgreSQL array column** (simpler, good for up to ~100 tags per row):

```
ALTER TABLE posts ADD COLUMN tag_ids INT[];
CREATE INDEX ON posts USING GIN (tag_ids);
-- Query: WHERE tag_ids @> ARRAY[1, 5, 10] (contains all these tags)
```

3. **JSONB column** (flexible, for tag metadata):

```
ALTER TABLE posts ADD COLUMN tags JSONB; -- [{"id":1,"name":"sql"},
{"id":2,"name":"postgres"}]
CREATE INDEX ON posts USING GIN (tags);
```

When to use each: M:N for normalized analytics and tag management; arrays for fast containment queries and smaller data sets; JSONB for arbitrary tag metadata.

Q7: How do you handle hierarchical data (categories, org charts) in PostgreSQL?

Core answer: Four approaches:

1. **Adjacency list** (simplest): `parent_id REFERENCES self`. Simple to update; recursive queries required (`WITH RECURSIVE`).
2. **Closure table**: A `category_ancestors(ancestor_id, descendant_id, depth)` table. $O(1)$ subtree and path queries. Expensive on updates.
3. **Materialized path (ltree extension)**: `path LTREE` column stores `'Electronics.Computers.Laptops'`. Fast subtree queries with `<@` operator; requires update on moves.
4. **Nested sets**: `lft INT, rgt INT`. $O(1)$ subtree reads; expensive updates.

Best for PostgreSQL: Use the `ltree` extension with materialized paths for read-heavy hierarchies (categories, menus) and adjacency list with `WITH RECURSIVE` for write-heavy hierarchies (org charts, comment trees).

```
-- ltree example
SELECT * FROM categories WHERE path <@ 'Electronics.Computers';
-- Returns all descendants of Electronics.Computers
```

Indexing and Performance Questions

Q8: Explain the difference between B-Tree, Hash, GIN, and GiST indexes.

Index Type	Best For	Notes
B-Tree	Equality, range, ORDER BY, LIKE 'prefix%'	Default; covers 99% of use cases
Hash	Equality only	Faster than B-Tree for pure equality; no range support; not WAL-logged before PG10
GIN	Full-text search, array containment, JSONB	Slow to update; use for rarely-modified, frequently-searched data
GiST	Geometric/geospatial, range types, full-text	Lossy (may need heap recheck); supports exclusion constraints
BRIN	Monotonically increasing data (timestamps, IDs)	Very small; works by block range min/max; only useful when data is physically sorted
SP-GiST	Hierarchical data, phone numbers, ltree	Space-partitioned tree; good for non-balanced data distributions

Key interview point: GIN indexes store each token/element separately and invert the mapping (element → row list). A GIN index on `tags INT[]` can answer "find all rows containing tag 5" by directly looking up tag 5 in the index without scanning rows.

Q9: When would you use a partial index?

Core answer: A partial index indexes only rows matching a WHERE condition. Use it when:

1. You frequently query a small subset of a large table.
2. The unindexed rows would never appear in the query.

```
-- 95% of orders are 'completed'. Only 'active' orders need fast lookup:
CREATE INDEX idx_orders_active ON orders (user_id, created_at)
  WHERE status NOT IN ('completed', 'cancelled');

-- Alert management: only 'open' alerts are acted on:
CREATE INDEX idx_alerts_open ON fraud_alerts (severity, created_at)
  WHERE status = 'open';

-- Soft-delete pattern: only non-deleted rows queried:
CREATE INDEX idx_users_email ON users (email)
  WHERE deleted_at IS NULL;
```

Why: The partial index is smaller (fits in memory more easily), the planner can use it when the WHERE clause matches, and index maintenance is cheaper (fewer inserts/updates touch it).

Q10: What is an index-only scan and when does it happen?

Core answer: An index-only scan retrieves data from the index without visiting the heap (the actual table). It happens when all columns needed by the query are present in the index (either as index key columns or INCLUDE d columns).

```
-- Regular index
CREATE INDEX idx_orders_user ON orders (user_id, status);
-- Query: SELECT user_id, status FROM orders WHERE user_id = $1
-- This CAN use index-only scan (both columns are in the index)

-- Including non-key columns
CREATE INDEX idx_products_sku ON product_variants (sku) INCLUDE (price, title);
-- Query: SELECT sku, price, title FROM product_variants WHERE sku = $1
-- Index-only scan – no heap access needed
```

Caveat: Index-only scans still need to check the visibility map for recently modified pages (MVCC). After a VACUUM , the visibility map marks pages as clean, enabling true index-only scans. Monitor pg_stat_user_indexes.idx_blks_hit vs heap_blks_read .

Q11: How do you investigate a slow query?

Core answer: Five-step process:

1. **EXPLAIN ANALYZE:** EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) . Look for: sequential scans on large tables, bad row estimates, nested loop on high-cardinality joins, sort spilling to disk.
2. **Check pg_stat_statements :** Find the slowest queries by total time, not just max time.
3. **Check indexes:** SELECT * FROM pg_stat_user_indexes WHERE relname = 'mytable' — are the right indexes being used?
4. **Update statistics:** ANALYZE mytable — stale statistics cause bad plans.
5. **Tune work_mem :** Complex GROUP BY or sort operations spill to disk if work_mem is too low. Set per-session: SET work_mem = '256MB' before the heavy query.

Interview signal: Mention that the planner's row estimates in EXPLAIN ANALYZE are the first thing to check. If actual rows >> estimated rows, the plan is based on wrong data — ANALYZE will fix it.

Q12: What is index bloat and how do you fix it?

Core answer: Index bloat occurs when index pages contain dead entries from UPDATES and DELETES that haven't been vacuumed. Bloated indexes waste memory and slow down scans.

Diagnosis:

```
-- pg_bloat_check or manual:
SELECT schemaname, tablename, attname,
       n_dead_tup, n_live_tup,
       ROUND(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_pct DESC;
```

Fix:

- `VACUUM tablename` — removes dead tuples, marks pages for reuse. Non-blocking.
- `VACUUM FULL tablename` — rewrites table and all indexes. Requires `ACCESS EXCLUSIVE` lock (blocks reads and writes). Use during maintenance windows only.
- `REINDEX CONCURRENTLY indexname` — rebuilds index without locking. Added in PostgreSQL 12.

Prevention: Tune `autovacuum_vacuum_scale_factor` (default 20% → set to 1–2% for high-churn tables).

Scaling and Architecture Questions

Q13: When should you partition a table?

Core answer: Partition when:

1. Table exceeds ~100GB and most queries have a natural time range filter.
2. You want to purge old data instantly (`DROP TABLE partition` is instant vs. `DELETE` which is slow and leaves bloat).
3. `VACUUM` is too slow to run on the whole table.
4. You can guarantee the partition key appears in almost all queries.

When NOT to partition:

- Small tables (< 50GB) — partition overhead outweighs benefits.
- Queries don't filter by the partition key (partition pruning won't trigger).
- Frequent cross-partition aggregates (JOINing all partitions negates the benefit).

```
-- Partition by month — common pattern
CREATE TABLE events (
    event_id    UUID NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL,
    ...
    PRIMARY KEY (event_id, occurred_at)
) PARTITION BY RANGE (occurred_at);

-- Drop old partitions instantly (vs hours-long DELETE)
DROP TABLE events_2022_01;
```

Q14: Explain the read replica architecture for PostgreSQL.

Core answer: PostgreSQL streaming replication sends a continuous stream of WAL (Write-Ahead Log) records from the primary to one or more replicas. Replicas replay these records to stay in sync.

Architecture:

```
Writes → Primary PostgreSQL
Reads  → Read Replica 1 (reporting, analytics)
        → Read Replica 2 (API read traffic)
        → Read Replica 3 (backup + data science)
```

Replication lag: Replicas can lag the primary. Read your own writes from the primary or use synchronous replication (`synchronous_commit = remote_apply`) for zero lag.

Connection pooling: Applications should use separate connection strings for read vs. write operations. Tools: PgBouncer, Pgpool-II, or application-level pool separation.

Limitation: Read replicas cannot handle write traffic. All writes must go to the primary.

Q15: When would you choose sharding over partitioning?

Core answer:

- **Partitioning:** Single PostgreSQL instance; data divided across multiple physical locations on the same server. Does not scale write capacity.
- **Sharding:** Data distributed across multiple PostgreSQL instances (different servers). Scales both read and write capacity.

Choose sharding when:

- Write throughput exceeds what a single primary can handle (~10K–50K writes/sec)
- Data size exceeds what a single server can store (~10TB+ practical limit before costs/performance degrade)
- Geographic data locality is required (store EU users' data in EU)

Sharding strategies:

- **Range sharding:** e.g., `user_id < 1M` on shard 1, `1M–2M` on shard 2. Easy to add shards at the high end; hot spot risk.
- **Hash sharding:** `user_id % N` determines shard. Even distribution; requires resharding to add capacity.
- **Directory sharding:** A lookup table maps `entity_id` → `shard_id`. Most flexible; lookup table is a potential bottleneck.

PostgreSQL tools for sharding: Citus extension, Postgres-XL, manual application-layer sharding.

Q16: What is connection pooling and why is it essential?

Core answer: Each PostgreSQL connection uses ~5–10MB of memory and requires a backend process. At 1000 connections, that is 5–10GB just for connection overhead before any data is processed.

Connection poolers (PgBouncer, Pgpool-II) maintain a small pool of long-lived PostgreSQL connections and multiplex thousands of client connections onto them.

PgBouncer modes:

- **Session mode:** One backend per client for the session lifetime. Compatible with all features (prepared statements, advisory locks). Use for ORM compatibility.
- **Transaction mode:** One backend per transaction. Much higher multiplexing ratio. Requires stateless connections (no `SET`, no temporary tables, no session-level prepared statements).
- **Statement mode:** One backend per statement. Rarely used; incompatible with transactions.

Production recommendation: PgBouncer transaction mode for OLTP; session mode for ORMs that use prepared statements. Aim for 10–50 backend connections serving thousands of client connections.

Transactions and Consistency Questions

Q17: Explain the four PostgreSQL isolation levels and when to use each.

Level	Dirty Reads	Non-Repeatable	Phantom Reads	Write Skew	Use When
READ UNCOMMITTED	Possible (but not in PG)	Possible	Possible	Possible	Never in PostgreSQL (treated as READ COMMITTED)
READ COMMITTED	No	Possible	Possible	Possible	Default; analytics, reporting, most web requests
REPEATABLE READ	No	No	No in PG	Possible	Multi-row consistency needed (statement-level snapshot)
SERIALIZABLE	No	No	No	No	Fund transfers, ticket booking, inventory reservation

PostgreSQL Serializable Snapshot Isolation (SSI): PostgreSQL's SERIALIZABLE uses SSI — an optimistic approach that allows transactions to proceed and only aborts when a conflict that would produce a non-serializable outcome is detected. Much more concurrent than traditional 2PL-based serializable isolation.

Q18: What is a deadlock and how do you prevent it?

Core answer: A deadlock occurs when two transactions each hold a lock the other needs. PostgreSQL detects deadlocks via a background process and aborts one transaction (the "victim").

Prevention strategies:

1. **Always acquire locks in the same order** — if transaction A always locks table1 then table2, and transaction B does the same, no deadlock is possible.
2. **Acquire all locks at transaction start** — `SELECT ... FOR UPDATE` all needed rows before any writes.
3. **Use NOWAIT or SKIP LOCKED** — fail fast instead of waiting, allowing the application to retry.
4. **Keep transactions short** — the longer a lock is held, the more likely a deadlock.

```
-- Banking transfer: lock in consistent order
-- Always lock lower account_id first
SELECT balance FROM accounts WHERE account_id IN ($a, $b) ORDER BY account_id FOR UPDATE;
```

Q19: What is MVCC and how does PostgreSQL implement it?

Core answer: MVCC (Multi-Version Concurrency Control) allows readers and writers to never block each other. Every row version has `xmin` (transaction that created it) and `xmax` (transaction that deleted/updated it). A transaction sees a row if `xmin <= snapshot.xid` and `(xmax = 0 OR xmax > snapshot.xid)`.

Key consequences:

- `SELECT` never blocks on `UPDATE/DELETE` (it reads the old version)
- `UPDATE` creates a new row version (old version becomes dead tuple)

- Dead tuples must be cleaned up by `VACUUM`
- `VACUUM` can only remove rows that are no longer visible to any active transaction

Write-Ahead Log (WAL): PostgreSQL's durability mechanism. Every change is written to the WAL before being applied to data pages. On crash, PostgreSQL replays the WAL to restore consistency.

Q20: How do you implement optimistic vs. pessimistic locking?

Pessimistic locking — lock before reading:

```
BEGIN;
SELECT * FROM accounts WHERE account_id = $1 FOR UPDATE;
-- Blocks other transactions from modifying this row
UPDATE accounts SET balance = balance - $amount WHERE account_id = $1;
COMMIT;
```

Optimistic locking — check at write time:

```
-- Add version column
ALTER TABLE accounts ADD COLUMN version INT NOT NULL DEFAULT 0;

-- Read without lock
SELECT balance, version FROM accounts WHERE account_id = $1;

-- Update only if version hasn't changed
UPDATE accounts
SET balance = balance - $amount, version = version + 1
WHERE account_id = $1 AND version = $read_version;
-- If 0 rows updated: conflict detected → retry
```

When to use which:

- Pessimistic: high contention, data integrity critical (banking, inventory)
 - Optimistic: low contention, better throughput, acceptable retry logic (user profile updates, CMS)
-

Advanced PostgreSQL Features Questions

Q21: When would you use JSONB vs. a normalized table?

Core answer: Use JSONB when:

- Schema is genuinely unknown at design time (product attributes, event properties)
- Data is read as a whole object rather than filtered by individual fields
- The set of fields varies per row (EAV-replacement)
- You need flexible querying without schema migrations

Use normalized tables when:

- You frequently filter, sort, or aggregate on specific fields
- Data has strong referential integrity requirements
- Multiple tables need to reference the same data
- The schema is stable

JSONB gotchas:

- Cannot enforce foreign keys on JSONB values
 - Queries on deeply nested JSONB are less readable
 - GIN index on JSONB can become large
 - Cannot use column statistics for cardinality estimation
-

Q22: Explain materialized views and when to use them.

Core answer: A materialized view stores the result of a query physically on disk. Unlike a regular view (which re-executes the query on every access), a materialized view is a snapshot that must be explicitly refreshed.

```
CREATE MATERIALIZED VIEW product_stats AS
SELECT
    p.category_id,
    COUNT(*) AS product_count,
    AVG(pv.price) AS avg_price,
    MIN(pv.price) AS min_price,
    MAX(pv.price) AS max_price
FROM products p
JOIN product_variants pv ON pv.product_id = p.product_id
GROUP BY p.category_id;

-- Index on the materialized view
CREATE INDEX ON product_stats (category_id);

-- Refresh (can be run concurrently with CONCURRENTLY)
REFRESH MATERIALIZED VIEW CONCURRENTLY product_stats;
```

Use when:

- Query is expensive (seconds) but freshness of a few minutes is acceptable
- Data changes infrequently relative to read frequency
- Report generation, dashboards, pre-aggregated stats

CONCURRENTLY option: Refreshes without locking the view for reads. Requires a unique index on the materialized view.

Q23: What are window functions and when do you use them?

Core answer: Window functions compute a value for each row based on a "window" of related rows, without collapsing rows like GROUP BY.

```
-- Running total of sales per salesperson
SELECT
    salesperson_id,
    sale_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY salesperson_id
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

```

    ) AS running_total,
    RANK() OVER (PARTITION BY region ORDER BY total_sales DESC) AS rank_in_region,
    LAG(amount, 1) OVER (PARTITION BY salesperson_id ORDER BY sale_date) AS
prev_sale,
    NTILE(4) OVER (ORDER BY amount) AS quartile
FROM sales;

```

Common window functions:

- ROW_NUMBER() : Unique sequential number
- RANK() / DENSE_RANK() : Ranking with/without gaps
- LAG() / LEAD() : Access previous/next row
- SUM() / AVG() OVER: Running aggregates
- NTILE(n) : Divide into N buckets

Q24: How does table inheritance work in PostgreSQL?

Core answer: PostgreSQL supports table inheritance (INHERITS), where a child table inherits all columns from the parent.

```

CREATE TABLE vehicles (
    vehicle_id UUID PRIMARY KEY,
    make        VARCHAR(100),
    model       VARCHAR(100),
    year        SMALLINT
);

CREATE TABLE cars (
    doors SMALLINT,
    trunk_liters INT
) INHERITS (vehicles);

CREATE TABLE trucks (
    payload_tons NUMERIC,
    cab_type VARCHAR(50)
) INHERITS (vehicles);

```

Limitation: Foreign keys do not span inheritance hierarchies. A column on another table cannot reference vehicles and automatically include rows in cars and trucks .

Modern alternative: Prefer declarative table partitioning (PostgreSQL 10+) for the partition use case, and use composite type or JSONB columns for the "different attributes per subtype" use case. Table inheritance is largely a legacy feature in modern PostgreSQL design.

System Design with PostgreSQL Questions

Q25: How would you design a rate limiter using PostgreSQL?

Core answer: Fixed window rate limiter:

```

CREATE TABLE rate_limits (
    key          VARCHAR(200) NOT NULL,    -- e.g., 'user:123:api_calls'
    window_start TIMESTAMPTZ NOT NULL,
    count        INT NOT NULL DEFAULT 0,
    PRIMARY KEY (key, window_start)
);

-- Atomic increment and check
INSERT INTO rate_limits (key, window_start, count)
VALUES ($key, DATE_TRUNC('minute', now()), 1)
ON CONFLICT (key, window_start)
DO UPDATE SET count = rate_limits.count + 1
RETURNING count <= $limit AS allowed, count;

-- Cleanup job (delete old windows)
DELETE FROM rate_limits WHERE window_start < now() - INTERVAL '5 minutes';

```

Better alternative: Redis with INCR + EXPIRE is superior for rate limiting (sub-millisecond, atomic, auto-expires). PostgreSQL is acceptable for low-volume API rate limiting (< 10K RPS).

Q26: How do you implement a job queue in PostgreSQL?

Core answer: PostgreSQL's `SKIP LOCKED` makes it an excellent job queue:

```

CREATE TABLE jobs (
    job_id      UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    type        VARCHAR(100) NOT NULL,
    payload     JSONB NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    attempts    INT NOT NULL DEFAULT 0,
    max_attempts INT NOT NULL DEFAULT 3,
    run_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    locked_at   TIMESTAMPTZ,
    locked_by   VARCHAR(100),
    failed_at   TIMESTAMPTZ,
    error       TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Worker: claim a job atomically
BEGIN;
UPDATE jobs
SET status = 'processing',
    locked_at = now(),
    locked_by = $worker_id,
    attempts = attempts + 1
WHERE job_id = (
    SELECT job_id FROM jobs
    WHERE status = 'pending'
        AND run_at <= now()
        AND attempts < max_attempts

```

```

ORDER BY run_at ASC
FOR UPDATE SKIP LOCKED
LIMIT 1
)
RETURNING *;
COMMIT;

```

`SKIP LOCKED` skips rows locked by other workers, enabling lock-free parallel job processing without deadlocks. Libraries like `Que` (Ruby), `River` (Go), and `pg-boss` (Node.js) build on this pattern.

Q27: How do you implement full-text search in PostgreSQL?

Core answer:

```

-- Store the tsvector (computed once, indexed)
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR;

-- Weighted vector: title gets weight 'A', body gets 'C'
UPDATE articles
SET search_vector =
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', body), 'C');

-- GIN index on the vector
CREATE INDEX ON articles USING GIN (search_vector);

-- Auto-update via trigger
CREATE TRIGGER articles_search_update
BEFORE INSERT OR UPDATE ON articles
FOR EACH ROW EXECUTE FUNCTION tsvector_update_trigger(
    search_vector, 'pg_catalog.english', title, body
);

-- Search query with ranking
SELECT id, title,
    ts_rank(search_vector, query) AS rank,
    ts_headline('english', body, query, 'MaxFragments=2') AS snippet
FROM articles, to_tsquery('english', 'postgresql & indexing') query
WHERE search_vector @@ query
ORDER BY rank DESC LIMIT 20;

```

When to move to Elasticsearch: When you need fuzzy matching, multilingual stemming, complex relevance tuning, faceted search, or when the search index itself exceeds 20–30% of table size (GIN indexes are large).

Q28: How would you implement event sourcing with PostgreSQL?

Core answer: Event sourcing stores the sequence of events rather than the current state. Current state is derived by replaying events.

```

CREATE TABLE events (
    event_id          UUID NOT NULL DEFAULT uuid_generate_v4(),

```



```

    aggregate_type VARCHAR(100) NOT NULL, -- 'Order', 'Account'
    aggregate_id   UUID NOT NULL,
    sequence_number BIGINT NOT NULL,        -- monotonically increasing per
aggregate
    event_type     VARCHAR(200) NOT NULL,
    event_data     JSONB NOT NULL,
    metadata       JSONB,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (event_id, created_at),
    UNIQUE (aggregate_id, sequence_number) -- prevents out-of-order gaps
) PARTITION BY RANGE (created_at);

-- Optimistic concurrency: reject if last_sequence doesn't match
INSERT INTO events (aggregate_type, aggregate_id, sequence_number, event_type,
event_data)
SELECT 'Order', $order_id, COALESCE(MAX(sequence_number), 0) + 1, $event_type,
$event_data
FROM events WHERE aggregate_id = $order_id
HAVING COALESCE(MAX(sequence_number), 0) = $expected_sequence;

```

Projections: Rebuild current state by replaying events into a `read_models` table. Use a `checkpoints` table to track the last processed event per projection.

Behavioral/Situational Questions

Q29: Tell me about a time you improved a slow database query.

Answer framework:

1. **Situation:** Describe the system and the problem (query taking X seconds, affecting Y users).
2. **Investigation:** How you used EXPLAIN ANALYZE, pg_stat_statements, or monitoring.
3. **Root cause:** What you found (missing index, bad join order, sequential scan).
4. **Solution:** What you changed and why.
5. **Result:** Measured improvement.

Sample answer structure: "Our product listing page was taking 3 seconds at peak load. I ran EXPLAIN ANALYZE and found a sequential scan on the 50M-row products table due to a missing composite index. The query was filtering on `(category_id, is_active)` and ordering by `avg_rating DESC`. I added `CREATE INDEX ON products (category_id, is_active, avg_rating DESC)`. The query dropped from 3 seconds to 8ms. I then added a partial index `WHERE is_active = TRUE` to make it smaller since 90% of products are active."

Q30: How do you handle a schema migration in a production database with zero downtime?

Core answer: The expand-contract (or parallel-change) pattern:

1. **Add new column as nullable** (`ALTER TABLE ADD COLUMN` — fast, non-blocking)
2. **Backfill in batches** (UPDATE 10,000 rows at a time with a delay to avoid lock contention)
3. **Update application to write both old and new** (deploy)
4. **Verify new column is fully populated**
5. **Update application to read from new column** (deploy)

6. Add **NOT NULL constraint** (uses constraint validation — non-blocking in PostgreSQL 12+: `NOT VALID`, then `VALIDATE CONSTRAINT`)
7. Drop old column (deploy)

```
-- Step 1: Add nullable (instant)
ALTER TABLE orders ADD COLUMN new_status TEXT;

-- Step 2: Backfill in batches (run via script)
UPDATE orders SET new_status = status::TEXT WHERE id BETWEEN $start AND $end;

-- Step 6: Add NOT NULL without full scan
ALTER TABLE orders ADD CONSTRAINT orders_new_status_not_null
CHECK (new_status IS NOT NULL) NOT VALID;
-- Later:
ALTER TABLE orders VALIDATE CONSTRAINT orders_new_status_not_null;
```

Quick Reference: Key Numbers to Know

Metric	Value
PostgreSQL max connections (practical)	100–500 (PgBouncer extends to 10K+)
Typical write throughput (single primary)	5K–50K rows/sec depending on row size
Index size (B-tree)	~30–50% of indexed column data
GIN index size	50–200% of indexed data
VACUUM needed above	20% dead tuples (default autovacuum threshold)
Partition pruning works when	partition key is in WHERE clause
pg_notify payload limit	8,000 bytes
Max table size	32TB per table
Max row size	1.6TB (theoretical)
Max columns per table	1,600
Point lookup latency (indexed)	0.1–1ms
Sequential scan (100M rows)	30–300 seconds
COPY bulk insert speed	100K–500K rows/sec
WAL segment size	16MB (default)

Common Pitfalls to Avoid

1. Using `SELECT *` in production code — pulls unnecessary columns, prevents index-only scans.
2. Missing `LIMIT` on user-facing queries — allows full table scans in production.

3. **N+1 queries** — loading 100 items then querying for each item's details separately. Fix with JOINS or batch queries.
 4. **OR conditions preventing index use** — use `UNION ALL` instead, or ensure each condition is covered by an index.
 5. **Using NOT IN with subqueries** — use `NOT EXISTS` or `LEFT JOIN / IS NULL` instead; `NOT IN` is NULL-unsafe.
 6. **Forgetting to ANALYZE after bulk inserts** — stale statistics cause bad plans.
 7. **Using TRUNCATE inside a transaction expecting to see the effect** — TRUNCATE is auto-committed in some drivers.
 8. **SELECT count(*) FROM table** — extremely slow on large tables due to MVCC. Use `pg_stat_user_tables.n_live_tup` for approximate counts.
 9. **Assuming ORDER BY is stable without LIMIT** — always include a tiebreaker (e.g., `ORDER BY created_at, id`).
 10. **Not testing with production data volumes** — an index on 100 rows behaves very differently from one on 100M rows.
-

Cross-References

- Amazon case study: `01_amazon_marketplace_db.md`
- Netflix case study: `02_netflix_streaming_platform.md`
- Uber case study: `03_uber_ridesharing.md`
- Banking case study: `04_banking_platform.md`
- SaaS case study: `05_saas_product.md`
- Messaging case study: `06_messaging_system.md`
- System Design: `21_System_Design/` folder
- Interview framework: `21_System_Design/08_system_design_framework.md`